

HARDEN A CONTAINER IMAGE

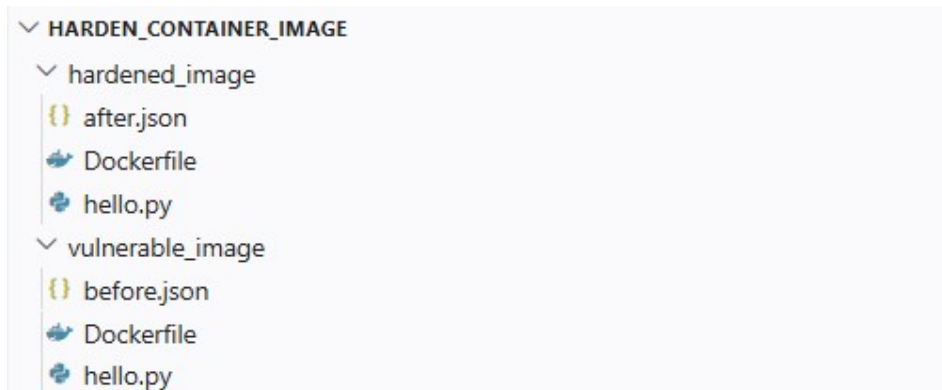
Introduction:

This practical project demonstrates how to harden a Docker container image, starting from a deliberately vulnerable base image running a Flask application and then applying DevSecOps hardening steps to reduce vulnerabilities.

Tools used:

- Docker
- Trivy for scanning the vulnerability of the container images
- Visual Studio Code

Project Structure:



Vulnerable image

Application code

```
1  from flask import Flask
2  app = Flask(__name__)
3
4  @app.route("/")
5  def hello():
6      return "Vulnerable image"
```

Dockerfile

```
FROM ubuntu:22.04

RUN apt-get update && apt-get install -y python3 python3-pip
RUN pip install flask==3.0.* --no-cache-dir

COPY hello.py /

ENV FLASK_APP=hello
EXPOSE 8000
USER root

CMD ["flask", "run", "--host", "0.0.0.0", "--port", "8000"]
```

This image is vulnerable because:

- it uses ubuntu:22.04 as the base image, which contains many OS packages
- running container as “root” allows potential attacker to compromise the host system
- it uses writable filesystem by default, which can lead to malware injection

Building the image:

```
docker build -t vulnerable_image:1 ./vulnerable_image
```

Running the image:

```
docker run --rm -p 8000:8000 vulnerable_image:1
```

Hardened image

Application code

```
from flask import Flask
app = Flask(__name__)

@app.route("/")
def hello():
    return "Hardened image container"
```

Dockerfile

```
FROM alpine:3.19

WORKDIR /app

RUN apk add --no-cache python3 py3-pip &&\
    pip install --break-system-packages flask==3.0.*

COPY hello.py .

RUN addgroup -S app &&\
    adduser -S appuser -G app

ENV FLASK_APP=hello
EXPOSE 8000
USER appuser

CMD ["flask", "run", "--host", "0.0.0.0", "--port", "8000"]
```

Starting from a minimal base image the following hardening steps are applied:

- minimal layers - chained instructions to prevent multiple layers
- least privilege principle - non-root user
- no package cache - to avoid storing index files on disk

Building the image

```
docker build -t hardened_image:1 ./hardened_image
```

Running the image - Read-Only

- prevents any data write attempts

```
docker run --rm --read-only --tmpfs /tmp -p 8000:8000 hardened_image:1
```

Vulnerability scanning using Trivy

```
docker run --rm \
-v /var/run/docker.sock:/var/run/docker.sock \
aquasec/trivy image -f json vulnerable_image:1 > vulnerable_image/before.json
```

```
docker run --rm \
-v /var/run/docker.sock:/var/run/docker.sock \
aquasec/trivy image -f json hardened_image:1 > hardened_image/after.json
```

Before and after CVE report comparison:

Severity	Vulnerable image	Hardened image
Critical	0	0
High	5	2
Medium	2005	5
Low	126	4
Unknown	0	0
Total	2136	11

Conclusion:

By implementing container hardening practices, the number of vulnerabilities dropped from 2136 to 11. This demonstrates that by applying changes such as: using a minimal base image, a non-root user, reducing installed packages and enabling a read-only filesystem the container security can be improved.

References:

<https://docs.docker.com/>

https://trivy.dev/docs/latest/guide/target/container_image/

https://hub.docker.com/_/alpine

https://hub.docker.com/_/ubuntu/

<https://botmonster.com/self-hosting/harden-docker-images-container-security-checklist/>

GitHub project:

https://github.com/andronlulia/harden_container_image